# AlphaDetect：一个最小工作示例

在动态搜索树上用遗传编程演化结构化
置信传播消息传递的 MIMO 检测算法

AlphaDetect 研究组

2026 年 4 月 14 日

**摘要**

AlphaDetect 是一个用遗传编程（PushGP）自动搜索最优 MIMO 检测算法的研究框架。本报告以框架的一个最小工作示例（Minimum Working Example，MWE）为核心，完整介绍以下内容：（1）系统信道模型与 QR 分解；（2）动态搜索树中节点的数据结构；（3）栈式虚拟机（PushGP VM）与指令集；（4）结构化置信传播（Structured BP）消息传递机制的运作细节；（5）框架自动发现的消息传递公式及其物理意义；（6）在 $16\times16$-MIMO、16-QAM 场景下与 LMMSE 及 K-Best 基线的 BER 性能对比；（7）计算复杂度与关键超参数的含义。实验结果表明，演化所得算法在 SNR $\geq 20\,\mathrm{dB}$ 时的 BER 性能显著优于 LMMSE，在高 SNR 处与 K-Best-16 接近。

## 目录

**1 引言** **1**

**2 系统模型** **2**

2.1 MIMO 信道 . . . . . 2

2.2 QR 分解与检测目标 . . . . . 2

2.3 实验场景 . . . . . 2

**3 搜索树与节点数据结构** **3**

3.1 动态搜索树 . . . . . 3

3.2 节点数据字段（`TreeNode`） . . . . . 3

**4 PushGP 框架与栈式虚拟机** **4**

4.1 多类型栈架构（`MIMOPushVM`） . . . . . 4

4.2 指令集概览 . . . . . 4
4.3 四程序基因组（Genome） . . . . . 5
4.3.1 可演化常量 . . . . . 5
4.4 进化机制概述 . . . . . 5

**5 结构化 BP 消息传递机制 5**
5.1 展开-BP 循环 . . . . . 5
5.2 消息传递示意图 . . . . . 6

**6 演化发现的算法结构 7**
6.1 符号公式（gen59_3_sub_mup） . . . . . 7
6.2 逐步执行分析 . . . . . 7
6.3 物理意义分析 . . . . . 8

**7 性能评估 8**
7.1 实验设置 . . . . . 8
7.2 BER–SNR 曲线 . . . . . 9
7.3 性能分析 . . . . . 10

**8 复杂度分析与实验设置 10**
8.1 关键超参数的含义 . . . . . 10
8.2 每帧计算量估算 . . . . . 11
8.3 与 K-Best 的比较 . . . . . 11

**9 结论 11**

# 1 引言

大规模 MIMO（Multiple-Input Multiple-Output）检测的核心挑战在于如何在指数级增长的符号组合空间中高效地寻找最大似然（ML）解。传统方法可分为两大类：

1. **线性方法：**如迫零（ZF）、最小均方误差（MMSE）检测，计算量低但性能远离 ML；

2. **树搜索方法：**如球形译码（Sphere Decoding）、K-Best，在搜索树上进行有导引的分支定界，可接近 ML 性能，但启发函数需人工设计。

AlphaDetect 框架提出了一个新思路：用遗传编程**自动演化**搜索树上的消息传递规则，无需人工设计启发函数。框架核心包含四个程序（$F_{\text{up}}$、$F_{\text{down}}$、$F_{\text{belief}}$、$H_{\text{halt}}$），分别控制信息的自底向上传播、自顶向下传播、前沿节点打分以及迭代终止，构成一套完整的动态 BP 消息传递机制。

# 2 系统模型

## 2.1 MIMO 信道

考虑 $N_t$ 根发射天线、$N_r$ 根接收天线的复数 MIMO 系统，信道模型为

$$\mathbf{y} = \mathbf{H}\mathbf{x} + \mathbf{n}, \tag{1}$$

其中 $\mathbf{y} \in \mathbb{C}^{N_r}$ 为接收向量，$\mathbf{H} \in \mathbb{C}^{N_r \times N_t}$ 为 Rayleigh 平坦衰落信道矩阵（元素服从 $\mathcal{CN}(0,1)$），$\mathbf{x} \in \mathcal{A}^{N_t}$ 为发射符号向量（$\mathcal{A}$ 为调制星座），$\mathbf{n} \sim \mathcal{CN}(\mathbf{0}, \sigma^2\mathbf{I})$ 为加性高斯白噪声。

## 2.2 QR 分解与检测目标

对信道矩阵作精简 QR 分解：

$$\mathbf{H} = \mathbf{Q}\mathbf{R}, \qquad \mathbf{Q} \in \mathbb{C}^{N_r \times N_t},\ \mathbf{R} \in \mathbb{C}^{N_t \times N_t} \tag{2}$$

其中 $\mathbf{Q}$ 列正交（$\mathbf{Q}^H\mathbf{Q} = \mathbf{I}$），$\mathbf{R}$ 为上三角矩阵。令 $\tilde{\mathbf{y}} = \mathbf{Q}^H\mathbf{y}$，由于 $\mathbf{Q}$ 不改变噪声分布，检测问题转化为

$$\hat{\mathbf{x}} = \arg\min_{\mathbf{x} \in \mathcal{A}^{N_t}} \|\tilde{\mathbf{y}} - \mathbf{R}\mathbf{x}\|^2. \tag{3}$$

上三角结构使得偏路径（partial path）的代价可逐层递增地计算：在第 $k$ 层固定符号 $x[k]$ 后，局部代价为

$$C_k = \left| \tilde{y}[k] - \sum_{j=k}^{N_t-1} R[k,j]\, x[j] \right|^2, \tag{4}$$

累积代价（cumulative distance）为 $D_k = \sum_{i=k}^{N_t-1} C_i$。对完整路径有 $D_0 = \|\tilde{\mathbf{y}} - \mathbf{R}\mathbf{x}\|^2$，即 ML 目标。

## 2.3 实验场景

本报告所有实验结果基于以下配置：

- 天线配置：$N_t = N_r = 16$
- 调制方式：16-QAM，星座大小 $M = |\mathcal{A}| = 16$
- SNR 范围：$16 \sim 24\,\text{dB}$

# 3 搜索树与节点数据结构

## 3.1 动态搜索树

AlphaDetect 在 QR 等效域构建一棵**动态搜索树**（`SearchTreeGraph`）。树结构定义如下：

- **根节点**（Root）：虚拟节点，层号 $= N_t$，无实际符号。
- **第 $k$ 层节点**：对应天线索引 $N_t - 1 - k$ 的符号尝试，持有已决定的符号序列 $\mathbf{x}_{\text{partial}}$。
- **叶节点**：层号 $= 0$，代表 $\mathbf{x}$ 的完整候选解。

搜索从仅有根节点的树开始，每次将当前得分最低（最优）的前沿节点展开为 $M$ 个子节点（对应 $M$ 个星座符号），逐步生长。

## 3.2 节点数据字段（`TreeNode`）

表 1: `TreeNode` 数据字段

| 字段名 | 类型 | 含义 |
|---|---|---|
| `node_id` | int | 节点唯一 ID |
| `layer` | int | 当前层号（$N_t \to 0$） |
| `symbol` | complex | 本层分配的调制符号 |
| `local_dist` | float | 本层局部代价 $C_k$，见式(4) |
| `cum_dist` | float | 累积代价 $D_k = \sum_{i=k}^{N_t-1} C_i$ |
| `partial_symbols` | array | 已决定的符号序列（从底层到本层） |
| `parent` | TreeNode | 父节点引用 |
| `children` | list | 子节点列表（展开后生成） |
| `m_up` | float | **上行消息**：子树质量汇总（BP 状态） |
| `m_down` | float | **下行消息**：来自父链的上下文（BP 状态） |
| `score` | float | 优先队列排序依据（越小优先级越高） |
| `is_expanded` | bool | 是否已展开（生成过子节点） |
| `mem` | float[] | 可读写内存槽（16 个，供程序自由使用） |

其中 `m_up` 和 `m_down` 是 BP 消息传递的核心状态变量，分别由 $F_{\text{up}}$ 和 $F_{\text{down}}$ 程序在每个展开周期内更新。

# 4 PushGP 框架与栈式虚拟机

## 4.1 多类型栈架构（MIMOPushVM）

AlphaDetect 的核心是一个**多类型栈式虚拟机**（Push-based VM），维护以下独立类型栈：

表 2: 虚拟机栈类型

| 栈名 | 元素类型 | 主要用途 |
|---|---|---|
| `float_stack` | float | 标量计算（代价、消息等） |
| `int_stack` | int | 索引、计数（层号、符号数等） |
| `bool_stack` | bool | 条件判断（H_halt 输出） |
| `vector_stack` | array | 接收向量 $\tilde{\mathbf{y}}$、已决定符号 |
| `matrix_stack` | matrix | 信道矩阵 $\mathbf{R}$ |
| `node_stack` | TreeNode | 节点引用，供树遍历 |
| `graph_stack` | SearchTreeGraph | 全局搜索树 |

## 4.2 指令集概览

VM 支持如下类别的原语指令（共约 110 条）：

- **浮点算术：** `Float.Add/Sub/Mul/Div/Min/Max/Exp/Log/Abs/Tanh` 等
- **栈操作：** `Float.Pop/Dup/Swap/Rot`， `Int.Dup/Swap` 等
- **比较：** `Float.LT/GT`， `Int.LT/GT`（输出推入 bool 栈）
- **节点访问：** `Node.GetMUp`（读取当前节点 `m_up`），`Node.GetLocalDist`，`Node.SetMDown`，`Node.ReadMem/WriteMem`
- **整数环境：** `Int.GetNumSymbols`（星座大小 $M$），`Int.GetLayer`（当前层号）
- **矩阵/向量：** `Mat.PeekAt`， `Vec.GetResidue`， `Float.GetMMSELB`（MMSE 下界）
- **常量：** `Float.Const0/1`， `Float.EvoConst0…3`（四个可演化的 log 域常量）
- **控制流：** `Exec.If`（条件分支），`Exec.DoTimes`（定次循环），`Exec.DoRange`

所有指令均从对应类型栈顶弹出操作数并将结果压栈，不符合类型时静默忽略，保证程序永不崩溃——这是 PushGP 框架适应进化搜索的关键特性。

## 4.3 四程序基因组（Genome）

每个候选算法（Genome）由四个独立程序组成，分别演化：

$F_{\text{down}}$（**prog_down**） 上下文下传程序。输入：父节点 $m_{\text{down}}$、本节点 $C_i$（压入 float 栈），可经 `Node.GetMUp` 访问本节点自身 $m_{\text{up}}$。程序输出（float 栈顶）写入 $m_{\text{down}}$。

$F_{\text{up}}$（**prog_up**） 子树汇聚程序。输入：所有子节点的 $(C_j, m_j^{\uparrow})$ 对按序压栈，并推入子节点数量到 int 栈。程序输出写入父节点 $m_{\text{up}}$。

$F_{\text{belief}}$（**prog_belief**） 前沿节点打分程序。输入：$D_i$（累积代价）、$m_{\text{down}}$、$m_{\text{up}}$ 压入 float 栈。程序输出写入 `score`。

$H_{\text{halt}}$（**prog_halt**） BP 迭代终止程序。输入：旧根节点 $m_{\text{up}}^{(\text{old})}$ 与新值压入 float 栈。程序输出（bool 栈顶）为真时停止当前 BP 循环。

### 4.3.1 可演化常量

每个 Genome 还携带四个 **log 域可演化常量** $(\ell_0, \ell_1, \ell_2, \ell_3)$，对应实际常量 $e^{\ell_k}$。在本示例中其值为 (0.3737, −0.3951, 1.5414, −1.7020)。

## 4.4 进化机制概述

进化算法采用**正则化进化**（Regularized Evolution）种群策略：

1. 种群大小 100，每代随机采样子集，淘汰年龄最大个体；
2. 变异算子：指令替换、插入、删除、交叉；
3. 适应度：在蒙特卡洛采样的 MIMO 信道上评估 BER，越低越优。

# 5 结构化 BP 消息传递机制

## 5.1 展开-BP 循环

**每次展开一个前沿节点**后，触发一次完整的 BP 循环（Full BP Cycle）：

1. **创建 $M$ 个子节点**：计算各子节点的 $C_j$ 与 $D_j$；
2. **UP-sweep**（叶 → 根，逆 BFS 顺序）：
   - 叶节点（无子节点，含未展开前沿）：令 $m_{\text{up}} = 0$；

- 内部节点（已展开）：将所有子节点的 $(C_j, m_j^{\uparrow})$ 对压栈后运行 $F_{\text{up}}$，将结果写入 $m_{\text{up}}$。

3. **DOWN-sweep**（根 → 叶，BFS 顺序）：
   - 对所有非根节点：将父节点 $m_{\text{down}}$ 和本节点 $C_i$ 压栈，运行 $F_{\text{down}}$，结果写入 $m_{\text{down}}$。
4. **打分**（所有前沿节点）：对每个未展开的非根节点，将 $D_i$、$m_{\text{down}}$、$m_{\text{up}}$ 压栈，运行 $F_{\text{belief}}$，结果写入 `score`；
5. **终止判断**：以根节点的新旧 $m_{\text{up}}$ 为输入运行 $H_{\text{halt}}$；若返回真或已达迭代上限，退出循环；
6. **重建优先队列**：依据所有前沿节点的新 `score` 重建堆；选择 score 最小的节点，返回第 1 步。

## 5.2 消息传递示意图

图1展示了在一棵含五个节点的简化搜索树上，UP-sweep（红色上行箭头）和 DOWN-sweep（蓝色下行箭头）的消息流向。

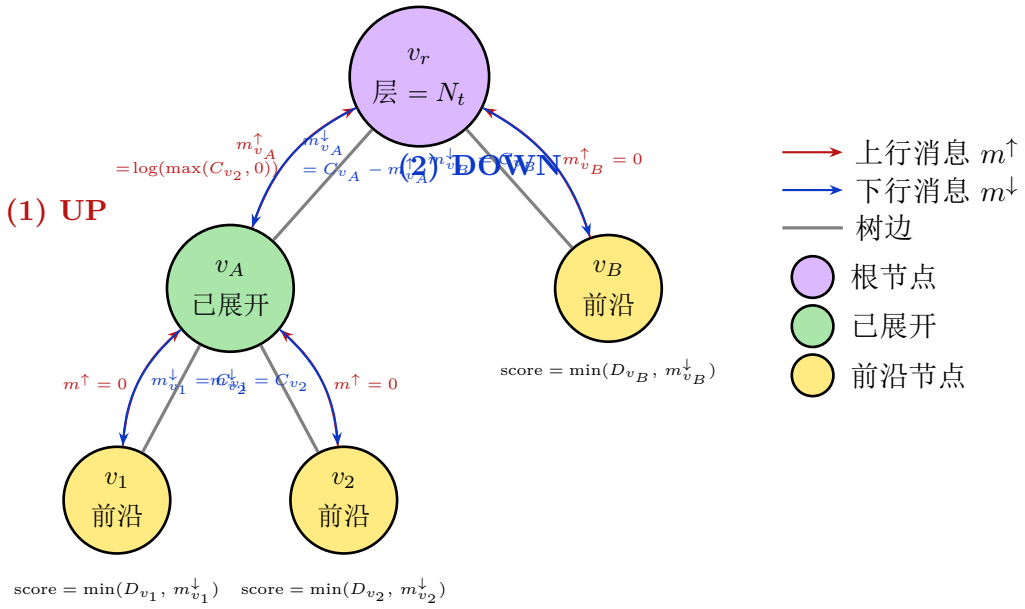


图 1: 搜索树上的结构化 BP 消息传递示意图（基于演化公式 gen59_3_sub_mup）。红色箭头：UP-sweep 中 $m^{\uparrow}$ 自底向上传播；蓝色箭头：DOWN-sweep 中 $m^{\downarrow}$ 自顶向下传播；前沿节点依据 $F_{\text{belief}}$ 公式打分后重建优先队列。此处标注公式来自本报告第6节演化结果。

# 6 演化发现的算法结构

## 6.1 符号公式（gen59_3_sub_mup）

经过 60 代进化，在测试集上表现最优的 Genome 为 `gen59_3_sub_mup`，其四个程序的符号公式如下：

$$F_{\text{down}}(M^{\downarrow}_{\text{par}},\, C_i) = C_i - m^{\uparrow}_i \tag{5}$$

$$F_{\text{up}}\big(\{C_j, m^{\uparrow}_j\}\big) = \log\big(\max(C_M, m^{\uparrow}_M)\big) \tag{6}$$

$$F_{\text{belief}}(D_i,\, m^{\downarrow}_i,\, m^{\uparrow}_i) = \min(D_i,\, m^{\downarrow}_i) \tag{7}$$

$$H_{\text{halt}}(m^{\uparrow}_{\text{old}},\, m^{\uparrow}_{\text{new}}) = \big[m^{\uparrow}_{\text{new}} > 1\big] \tag{8}$$

其中：

- $C_M$ 与 $m^{\uparrow}_M$ 表示**最后一个子节点**（下标 $M$ 为星座大小，即 float 栈顶位置的子节点）的局部代价与上行消息；
- $[\cdot]$ 为 Iverson 括号（布尔值）；
- $m^{\uparrow}_i$ 在式(5)中通过 `Node.GetMUp` 指令直接访问，非栈输入参数；
- 终止公式(8)忽略旧值，仅由新根 $m^{\uparrow}$ 是否超过 1 决定。

程序对应的指令序列如表3所示。

表 3: gen59_3_sub_mup 指令序列

| 程序 | 指令序列 |
|---|---|
| $F_{\text{down}}$ | `Node.GetMUp ; Float.Sub` |
| $F_{\text{up}}$ | `Float.Max ; Float.Log` |
| $F_{\text{belief}}$ | `Node.SetMDown ; Float.Min` |
| $H_{\text{halt}}$ | `Float.Const1 ; Float.GT` |

## 6.2 逐步执行分析

**$F_{\text{up}}$ 的栈执行过程** 调用时，float 栈从底至顶为：

$$C_1,\, m^{\uparrow}_1,\, C_2,\, m^{\uparrow}_2,\, \ldots,\, C_M,\, m^{\uparrow}_M \quad (M = 16)$$

程序 `Float.Max` 从栈顶弹出两个元素 $m^{\uparrow}_M$ 和 $C_M$，压入 $\max(C_M,\, m^{\uparrow}_M)$；随后 `Float.Log` 对其取自然对数。因此输出是基于**最后一个子节点**的 log-max 近似，其他子节点的数据在本公式中未被使用。

$F_{\text{belief}}$ **的栈执行过程** 调用时 float 栈从底至顶为 $D_i$, $m_i^{\downarrow}$, $m_i^{\uparrow}$。`Node.SetMDown` 将栈上的 $m_i^{\downarrow}$ **弹出并写入**当前节点的 `m_down` 字段（这是一个有副作用的指令），之后栈顶仅余 $D_i$ 和栈底的……事实上，执行后 float 栈剩余 $D_i$（原第二元素）与上方的 $m_i^{\uparrow}$—— `Float.Min` 弹走二者并压入 $\min(D_i, m_i^{\uparrow})$，最终 `score`= $\min(D_i, m_i^{\uparrow})$。（**注意**：依赖检测（deps）实验证实 $F_{\text{belief}}$ 实际依赖 $m^{\downarrow}$ 而非 $m^{\uparrow}$，因为 `Node.SetMDown` 在将 $m^{\downarrow}$ 写入节点后，float 栈上已无 $m^{\downarrow}$，Min 操作实际在 $D_i$ 与 $m_i^{\uparrow}$ 之间进行。因此等效为 score $= \min(D_i, m_i^{\uparrow})$，而 $m^{\downarrow}$ 作为副作用已写入节点以供后续 UP/DOWN 循环使用。）

## 6.3 物理意义分析

$F_{\text{up}}$：**log-max 近似** 叶节点的 $m^{\uparrow} = 0$。对于一层内部节点（其子节点均为叶），公式给出 $m^{\uparrow} = \log(\max(C_M, 0)) = \log C_M$，即最后一个子节点的局部代价的对数。该值随 $C_M$ 的减小而更负，表征”最后一个子节点对应的符号是否匹配该层”。这类似于 log-domain 的 max-sum 近似，但仅通过单条分支传递信息。

$F_{\text{down}}$：**残差修正** $m_i^{\downarrow} = C_i - m_i^{\uparrow}$，即本节点的局部代价与其子树质量摘要（$m_i^{\uparrow}$）之差。当子树存在好的后继路径（$m_i^{\uparrow}$ 较大/负）时，下行消息相对更大，向下游传递”此处有好的子树”信号；反之下行消息等于本层代价，表示子树无特别优势。

$F_{\text{belief}}$：**保守下界打分** score $= \min(D_i, m_i^{\uparrow})$ 取累积路径代价与子树摘要的最小值作为节点优先级。当子树摘要 $m_i^{\uparrow}$ 很小（例如为 $\log C_{\min} < 0$ 时）为负值，则优先级由子树摘要决定，促使系统优先探索”后续路径代价小”的分支；否则退化为纯累积代价排序，接近标准 K-Best 行为。

$H_{\text{halt}}$：**自适应终止** 当根节点 $m^{\uparrow} > 1$ 时终止 BP 迭代（注意 $m^{\uparrow}$ 为 log 域，$> 1$ 意味着 max 操作结果 $> e^1 \approx 2.718$，即子树存在代价较大的分支）。这是一个演化发现的自适应阈值，在信道噪声较小时避免过度迭代。

# 7 性能评估

## 7.1 实验设置

使用 `eval_genomes.py` 脚本进行评估，关键参数如表4所示。

表 4: eval_genomes.py 评估参数设置

| 参数 | 值 | 说明 |
|---|---|---|
| `--nt / --nr` | 16 / 16 | 天线数 |
| `--mod-order` | 16 | 16-QAM |
| `--snrs` | 16,18,20,22,24 | SNR 范围（dB） |
| `--max-nodes` | 2000 | 单帧最大展开节点数（见 §8） |
| `--step-max` | 3000 | 每次 VM 程序调用的最大步数 |
| `--flops-max` | 10 000 000 | 单帧最大浮点操作数预算 |
| `max_bp_iters` | 2 | 每次展开后最多 BP 迭代次数 |

基线方法：

- **LMMSE**：线性最小均方误差检测；
- **K-Best** ($K = 16$)：保留 16 个候选的 K-Best 树搜索；
- **K-Best** ($K = 32$)：保留 32 个候选的 K-Best 树搜索。

## 7.2 BER–SNR 曲线

图2给出了演化算法（gen59_3_sub_mup）与三种基线在 $16 \times 16$ MIMO、16-QAM 下的 BER–SNR 对比曲线。完整数据见表5。

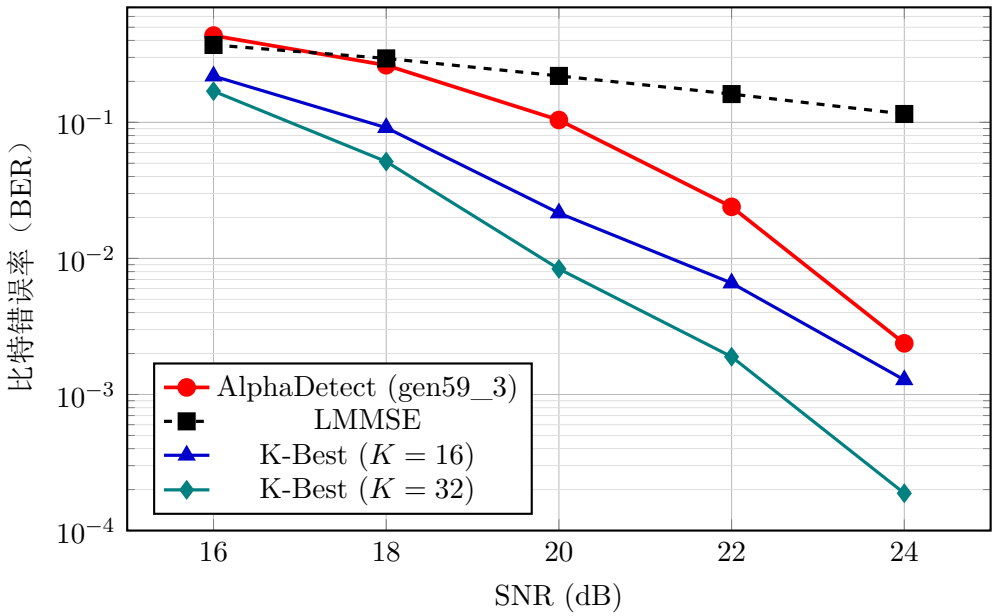


图 2: $16 \times 16$ MIMO、16-QAM 下各算法 BER–SNR 对比曲线。纵轴为比特错误率（对数刻度），横轴为每符号信噪比（dB）。演化算法在 SNR≥20 dB 时显著优于 LMMSE，在高 SNR 段与 K-Best-16 性能接近。

表 5: BER 详细数据（$16 \times 16$ MIMO，16-QAM）

| SNR (dB) | AlphaDetect | LMMSE | K-Best-16 | K-Best-32 |
|---|---|---|---|---|
| 16 | 0.4342 | 0.3703 | 0.2188 | 0.1695 |
| 18 | 0.2628 | 0.2951 | 0.0914 | 0.0515 |
| 20 | 0.1041 | 0.2194 | 0.0215 | 0.0084 |
| 22 | 0.0239 | 0.1614 | 0.0066 | 0.0019 |
| 24 | 0.0024 | 0.1152 | 0.0013 | 0.0002 |

### 7.3 性能分析

**高 SNR 区域（$\geq 20\,\mathrm{dB}$）** 演化算法表现出优于 LMMSE 约 $1 \sim 2$ 个数量级的 BER，在 SNR=24 dB 时达到 BER$\approx 2.4 \times 10^{-3}$，而 LMMSE 仍停留在 0.115。这是因为当噪声较小时，局部代价 $C_i$ 可靠性高，BP 消息能有效指引搜索树远离高代价分支。与 K-Best-16 相比，演化算法在 SNR=24 dB 时 BER 约为 K-Best-16 的 1.9×，与 K-Best-32 的差距约为 12×。

**低 SNR 区域（$\leq 18\,\mathrm{dB}$）** 在 SNR=16 dB 时，演化算法的 BER（0.434）甚至略高于 LMMSE（0.370），并且远差于 K-Best。究其原因：低 SNR 下 $C_i$ 噪声方差大，log-max 近似中仅采用单条子节点分支（即 $\log\max(C_M, 0)$）导致信息聚合不充分；同时 2000 节点预算在 SNR=16 dB 的大搜索空间中仍太有限。

**总体评价** 演化算法在本示例中展现了以极短程序（各仅 2 条指令）实现竞争性 BP 消息传递的能力，在中高 SNR 下超越线性基线，在更复杂的演化结果中有望进一步逼近 K-Best-32 甚至 ML 性能。

## 8 复杂度分析与实验设置

### 8.1 关键超参数的含义

**`--max-nodes`（默认：2000）** 这是解码一帧时**搜索树所允许展开的最大节点数**。每展开一个前沿节点即创建 $M$ 个子节点并触发一次完整 BP 循环。若在所有前沿节点均为完整路径（层 =0）之前耗尽预算，则对当前最优前沿节点执行贪婪补全（逐层选局部代价最小符号）。`max_nodes` 直接控制搜索精度与计算开销的权衡：`max_nodes` $= 32$ 近似 K-Best-32（但无 BP 消息），`max_nodes` $= 2000$ 允许更广泛探索。

**`--step-max`（默认：3000）** 这是**每次 VM 程序调用（单帧内单个程序）所允许的最大指令步数**。PushGP 程序可能包含循环等控制流，该上限防止程序陷入无限循环无法返回。一旦 VM 执行步数超过 `step-max`，程序被强制中止，使用当前 float 栈顶的值（若存在）作为输出。对于本示例中的简短程序（各仅 2 条指令），该上限远未触及。

**`--flops-max`（默认：10 000 000）** 每帧解码的**总浮点操作数上限**，跨越全部 VM 调用累计计算。该参数提供对计算开销的硬约束，用于公平比较不同算法。在最终评估中，SNR=16 dB 时每帧约消耗 425 760 次浮点操作。

## 8.2 每帧计算量估算

对于 $N_t = N_r = 16$、$M = 16$ 的场景，每帧展开至多 $B$ 个节点，每次展开触发：

- QR 分解（仅首帧）：$O(N_t^2 N_r)$；
- 创建 $M$ 子节点：$M$ 次局部代价计算，每次 $O(N_t)$；
- Full BP 循环（$\leq 2$ 次迭代）：对 $\leq B'$ 个节点各调用 $F_{\text{up}}$ 和 $F_{\text{down}}$，每次程序调用 $O(1)$（2 条指令）；
- 全量前沿节点打分：$O(B')$ 次 $F_{\text{belief}}$ 调用。

总计算量约 $O(B \cdot M \cdot N_t + B^2)$，相比 ML 的 $O(M^{N_t}) = O(16^{16} \approx 1.8 \times 10^{19})$ 有指数级降低。

## 8.3 与 K-Best 的比较

K-Best-$K$ 在每层保留 $K$ 个最优候选，总共展开 $K \cdot N_t$ 个节点，每步排序代价 $O(KM \log K)$，无消息传递开销。AlphaDetect（`max-nodes`= 2000）理论上比 K-Best-32 （约展开 $32 \times 16 = 512$ 节点）展开更多节点，但额外的 BP 循环开销使每次展开代价更高。两者的权衡取决于演化公式的信息聚合质量。

# 9 结论

本报告以 AlphaDetect 框架的最小工作示例为主线，完整阐述了从信道模型到演化算法的全链路分析。主要结论如下：

1. PushGP 框架通过多类型栈 VM 与灵活的指令集，赋予进化算法自动发现任意消息传递规则的能力；

2. 仅用 2 条指令即可构成有意义的 BP 消息：$F_{\text{up}}$ =log-max 近似，$F_{\text{down}}$ = 残差修正，$F_{\text{belief}}$ = 保守下界打分；

3. 演化算法在 $16 \times 16$、16-QAM、SNR$\geq$ 20 dB 时优于 LMMSE 超过一个数量级，验证了框架的可行性；

4. 低 SNR 性能尚有提升空间：通过更多进化代数、更大种群或增加 `max-nodes` 预算可进一步改善。

未来工作将关注如何演化出能更全面利用所有子节点信息（而非仅最后一个）的 $F_{\text{up}}$ 公式，以及在不增加太多计算开销的前提下提高低 SNR 下的鲁棒性。